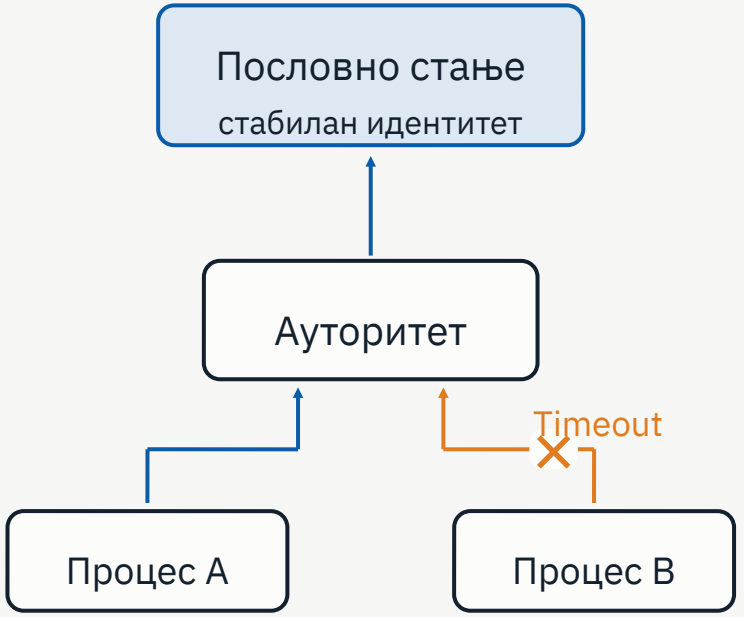
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Начин размишљања у дистрибуираном систему

Власништво и неизвесност пре мрежне комуникације



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–20

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

21–28

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Разумети како време, мрежа и више процеса мењају начин на који систем доноси одлуке.

Компоненте не виде исто стање у истом тренутку. Веза може нестати, одговор закаснити, а процес се може поново појавити.

> НАПОМЕНА
> Јасно власништво над стањем и операцијом спречава сукоб више инстанци и даје смисао статусима које систем приказује.

01 · ПОЈМОВИ

Четири појма теме

Дистрибуирани систем није обична апликација распоређена на више процеса.

01 **Пословни ентитет**

стабилан кроз време

02 **Чвор или процес**

привремена извршна инстанца

03 **Веза**

може каснити или нестати

04 **Власништво**

јасна надлежност

Пословни ентитет

Пословни ентитет, попут налога или уређаја, остаје исти и када се процеси рестартују. Његов идентитет не сме зависити од једне извршне инстанце.

01 Идентитет не зависи од једне извршне инстанце.

02 Преживљава рестарт процеса који га обрађује.

03 Пример: мисија или станица, не worker који је тренутно опслужује.

01 · ПОЈМОВИ

Процес

Процес је привремени извршилац који може нестати и поново се појавити. Зато његово локално стање није аутоматски извор истине за цео систем.

01 Привремени извршилац, не извор истине.

02 Може нестати и поново се појавити.

03 Локално стање процеса није глобално стање система.

01 · ПОЈМОВИ

Мрежна веза

Мрежна веза може каснити, прекинути се или вратити без јасне границе када се то десило. Систем мора рачунати на ту неизвесност.

01 Кашњење и прекид су очекивано понашање, не изузетак.

02 Нема јасне границе када се прекид тачно десио.

03 Систем мора дизајнирати око те неизвесности.

Власништво

Власништво одређује која компонента има право да донесе важећу одлуку о стању. Без њега две инстанце лако направе сукобљене измене.

01 Одређује ко сме да донесе важећу одлуку о стању.

02 Спречава да две инстанце сукобљено мењају исто стање.

03 Мора бити експлицитно, не претпостављено.

Пример: прекид везе није нестанак стања

Централна компонента престаје да прима heartbeat од удаљеног чвора.

01 Статус треба да изрази последње поуздано сазнање система.

02 Поновно повезивање не сме аутоматски значити да је настала нова пословна инстанца.

03 Промена власништва над радом мора имати правило које спречава стари чвор да настави са радом.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема ауторитета и неизвесности

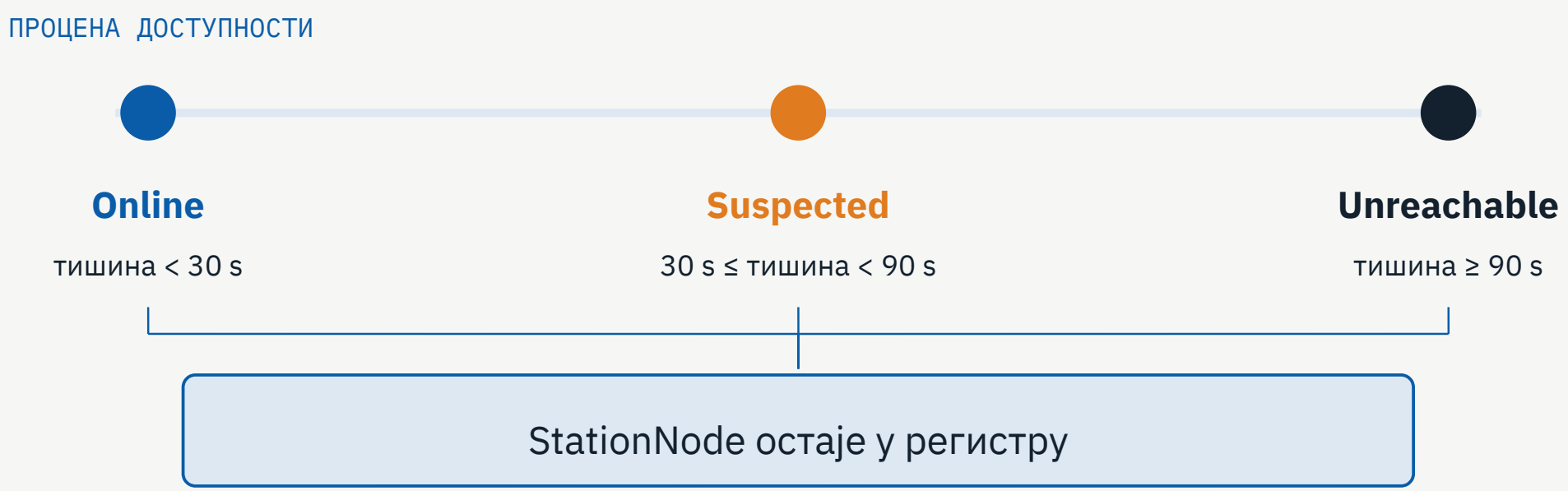
Процес и веза могу нестати, али пословно стање захтева јасан и стабилан ауторитет.

Елемент	Улога у току
Пословно стање	стабилна чињеница система
Ауторитет	компонента која одлучује
Мрежа	потврда може каснити
Видљив статус	оно што систем поуздано зна

02 · ДИЈАГРАМ

Тишина мења процену, а не идентитет

Пример из кода: прагови су 30 s и 90 s, док исти чвор остаје познат систему.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример раздваја пословни ентитет од процеса који га представља.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/01 Vlasnistvo neizvesnost"
dotnet run --project src/Odp.Vezba01.ConsoleUi
dotnet test Odp.Vezba01.sln
```

Правило у коду

Статус зависи само од дужине тишине, разлог тишине систем не зна и не претпоставља.

C# · доменско правило

```
1   public NodeLivenessAssessment Evaluate(StationNode node, DateTimeOffset now)
2   {
3       ArgumentNullException.ThrowIfNull(node);
4       var silence = SilenceOf(node, now);
5       if (silence < _thresholds.SuspectAfter)
6           return new(NodeStatus.Online, NodeLivenessCodes.HeartbeatFresh, silence);
7       if (silence < _thresholds.UnreachableAfter)
8           return new(NodeStatus.Suspected, NodeLivenessCodes.HeartbeatLate, silence);
9       return new(NodeStatus.Unreachable, NodeLivenessCodes.HeartbeatMissing, silence);
10  }
11  private static TimeSpan SilenceOf(StationNode node, DateTimeOffset now) =>
12      now > node.LastHeartbeatAt ? now - node.LastHeartbeatAt : TimeSpan.Zero;
```

Тест који задаје квар

Чвор који је Unreachable остаје у регистру: нестанак процеса није нестанак ентитета.

C# · NUnit

```
[Test]
public void Assess_WhenNodeStaysSilent_ReportsItUnreachableButKeepsItRegistered()
{
    var clock = new ManualClock(Start);
    var registry = new InMemoryNodeRegistry([new StationNode("node-ns-1", "GS-NOVI-SAD", Start)]);
    var handler = new MonitorNodesHandler(
        registry,
        new NodeLivenessPolicy(new LivenessThresholds(TimeSpan.FromSeconds(30),
            TimeSpan.FromSeconds(90))),
        clock);
    clock.Advance(TimeSpan.FromMinutes(10));
    var view = handler.Assess().Single();
    using (Assert.EnterMultipleScope())
    {
```

Тест који задаје квар (наставак)

Чвор који је Unreachable остаје у регистру: нестанак процеса није нестанак ентитета.

C# · NUnit

```
15          Assert.That(view.Liveness.Status, Is.EqualTo(NodeStatus.Unreachable));
16          Assert.That(view.Liveness.Code, Is.EqualTo(NodeLivenessCodes.HeartbeatMissing));
17          Assert.That(registry.Find("node-ns-1"), Is.Not.Null);
18      }
19  }
```

Испис демонстрације

Закаснели heartbeat добија код heartbeat_out_of_order и не враћа познато стање уназад.

console · исходи демонстрације

```
POČETAK [09:00:00]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
  heartbeat node-ns-1 poslat u 09:00:45 -> heartbeat_accepted
POSLE 45 s (javio se samo node-ns-1) [09:00:45]
  node-bg-1 (GS-BEOGRAD): Suspected | heartbeat_late | tišina 45 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
POSLE 105 s [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-ns-1 poslat u 09:00:20 -> heartbeat_out_of_order
STIGAO ZAKASNELI HEARTBEAT ZA node-ns-1 [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
```

Испис демонстрације (наставак)

Закаснели heartbeat добија код heartbeat_out_of_order и не враћа познато стање уназад.

console · исходи демонстрације

```
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-bg-1 poslat u 09:01:45 -> heartbeat_accepted
node-bg-1 SE PONOVO JAVIO [09:01:45]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Процес, мрежна веза и пословни ентитет нису иста ствар.
- Timeout значи да одговор није стигао на време, не да се операција сигурно није десила.
- Статус треба да изрази оно што систем зна, а не оно што претпоставља.

ЗАМКЕ

- Изједначавање прекида везе са трајним кваром.
- Две инстанце које без правила мењају исто стање.
- Пословно правило скривено у комуникационом детаљу.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Прекид везе се тумачи као трајни квар.

02 Две инстанце мењају исто стање без правила.

03 Пословно правило се сакрије у мрежни детаљ.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Описати ауторитет за важно стање.

02 Разликовати оно што систем зна од онога што претпоставља.

03 Тестирати привремени прекид као очекиван сценарио.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Које стање мора имати један јасан ауторитет и шта се види када тај ауторитет привремено није доступан?

> ЗА ВЕЖБУ
> Који податак или операција мора имати јасан ауторитет и како систем препознаје да је тај ауторитет привремено недоступан?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати статус Draining за чвор који је најавио планирано гашење.

02 Такав чвор не сме постати Unreachable док траје најављени период.

03 Правило додати у NodeLivenessPolicy, увести нов код и покрити га тестом.

04 · ПРИМЕНА

Примена на пројекат

Принцип са вежбе примењује се на додељену пројектну целину.

01 Идентификовати конкретну дистрибуирану одлуку у својој целини.

02 Имплементирати је са стабилним кодом исхода.

03 Припремити поновљив failure сценарио и тест који га доказује.

04 · ПРИМЕНА

Резиме

Тим најпре описује границе система, одговорност сваке компоненте и стање које је за њу ауторитативно.

Процес и веза могу нестати, али пословно стање захтева јасан и стабилан ауторитет.

НАПОМЕНА

Наредна вежба: Уговори, симулатори и поновљивост.